

ComicCraft Implementation Blueprint

AI Comic Story Creator using Gemini Models & Stable Diffusion — Complete Technical & Code Reference

1. Executive Project Overview

ComicCraft is a FastAPI-powered web application that automates the multi-modal generation of comic books. By combining **Google Gemini Flash** (structured planning), **Google Gemini Pro** (narrative/dialogue expansion), and **Stable Diffusion** via Diffusers (panel illustration generation), ComicCraft turns simple user text prompts into full 5-panel formatted comics accessible via browser or downloadable as structured PDFs.

2. Technical Architecture & File Structure

Component / Module	File Path	Responsibility
Main Entrypoint	app/main.py	Initializes FastAPI application, mounts static directories, sets up Jinja2 templates, and includes application routes.
Route Handlers	app/routes.py	Manages HTTP endpoints for form submission, JSON API execution, image testing, and preview rendering.
Comic Outline Generator	app/gemini_flash.py	Interacts with Gemini 1.5 Flash to generate a 5-panel JSON structured comic outline.
Story Scriptwriter	app/gemini_pro.py	Uses Gemini 1.5 Pro to expand the structural outline into vivid narration and character dialogue.
Illustration Generator	app/image_generator.py	Pipeline for rendering panel illustrations using Stable Diffusion pipeline and saving them locally.
Layout Assembly	app/layout_builder.py	Glues structured panel outlines, full story narration, and image paths into a consolidated dictionary layout.
PDF Exporting Service	app/exporters.py	Compiles multi-page PDF documents containing story panels, text, and artwork using FPDF.

3. Complete Application Code Files

requirements.txt

```
fastapi==0.104.1
uvicorn==0.24.0
jinja2==3.1.2
python-multipart==0.0.6
google-generativeai==0.3.1
diffusers==0.21.4
transformers==4.34.1
fpdf2==2.7.6
Pillow==10.1.0
accelerate==0.24.1
torch==2.1.0
```

```
pydantic==2.5.0
python-dotenv==1.0.0
```

app/main.py

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
import os

from app.routes import router

app = FastAPI(title="ComicCraft - AI Comic Story Creator")

# Create static directories if they do not exist
os.makedirs("static/panels", exist_ok=True)
os.makedirs("static/exports", exist_ok=True)

# Mount static files
app.mount("/static", StaticFiles(directory="static"), name="static")

# Include routes
app.include_router(router)

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("app.main:app", host="127.0.0.1", port=8000, reload=True)
```

app/gemini_flash.py

```
import json
import os
import google.generativeai as genai

# Configure Gemini API Key
genai.configure(api_key=os.getenv("GEMINI_API_KEY", "YOUR_GEMINI_API_KEY"))

model = genai.GenerativeModel('models/gemini-1.5-flash')

def generate_outline(user_prompt: str) -> list:
    """
    Generates a 5-panel comic layout based on the user's story idea using Gemini Flash.
    """
    prompt = f"""You are a professional AI comic planner.

Your task is to generate a *strictly formatted* JSON array containing 5 panel descriptions for a comic based
on the story idea below:

STORY: "{user_prompt}"

Each JSON object must include:
- "panel" (integer)
- "title" (string)
- "scene_description" (string)
- "image_prompt" (string)

Respond ONLY in this valid JSON format, without any explanations or markdown:
[
  {{
    "panel": 1,
    "title": "Title here",
    "scene_description": "Scene description here",
    "image_prompt": "Image prompt for Stable Diffusion"
  }}
]
"""
    try:
        response = model.generate_content(prompt)
        output_text = response.text.strip()
```

```

    if output_text.startswith("`json`"):
        output_text = output_text.replace("`json", "").replace("`", "").strip()

    panel_data = json.loads(output_text)

    if not isinstance(panel_data, list):
        raise ValueError("Gemini response is not a list.")

    for panel in panel_data:
        if not isinstance(panel, dict) or not all(k in panel for k in ("panel", "title",
"scene_description", "image_prompt")):
            raise ValueError(f"Invalid panel format: {panel}")

    return panel_data

except json.JSONDecodeError as e:
    return [{"error": f"JSON parsing failed: {str(e)}"}]
except Exception as e:
    return [{"error": f"Generation failed: {str(e)}"}]

```

app/gemini_pro.py

```

import os
import google.generativeai as genai

genai.configure(api_key=os.getenv("GEMINI_API_KEY", "YOUR_GEMINI_API_KEY"))
model = genai.GenerativeModel('models/gemini-1.5-pro')

def generate_story(outline: list) -> str:
    """
    Generates a detailed comic story with narration and character dialogue from an outline.
    """
    formatted_outline = "\n".join([f"Panel {item.get('panel')}: {item.get('title')} - {item.get('scene_description')}" for item in outline])

    prompt = f"""You're a comic book writer.

Given the following panel breakdown, write a comic-style story with engaging narration and character dialogues for each panel.

Panel Outline:
{formatted_outline}

Guidelines:
- Use a fun and engaging tone, like an actual comic book.
- Format each panel clearly starting with '**Panel 1**', '**Panel 2**', etc.
- Include narration and clearly marked character lines.
- Keep each panel self-contained but part of a cohesive story.
"""
    try:
        response = model.generate_content(prompt)
        return response.text
    except Exception as e:
        return f"Error generating story: {str(e)}"

```

app/image_generator.py

```

import os
import re
from PIL import Image

# Lazy load or mock pipeline if running on non-GPU environment
pipe = None

def get_pipeline():
    global pipe
    if pipe is None:
        try:

```

```

import torch
from diffusers import StableDiffusionPipeline
model_id = "runwayml/stable-diffusion-v1-5"
device = "cuda" if torch.cuda.is_available() else "cpu"
pipe = StableDiffusionPipeline.from_pretrained(model_id)
pipe = pipe.to(device)
except Exception as e:
    print(f"Warning: Stable Diffusion setup failed/skipped: {e}")
    pipe = "MOCK"
return pipe

def sanitize_filename(prompt: str) -> str:
    clean = re.sub(r'^a-zA-Z0-9_', '_', prompt[:30])
    return f"{clean}.png"

def generate_image(prompt: str, filename: str = None) -> str:
    if not filename:
        filename = sanitize_filename(prompt)

    path = f"static/panels/{filename}"
    os.makedirs(os.path.dirname(path), exist_ok=True)

    pipeline = get_pipeline()
    if pipeline != "MOCK" and pipeline is not None:
        image = pipeline(prompt).images[0]
        image.save(path)
    else:
        # Fallback dummy image creation for demo / low-resource setups
        img = Image.new('RGB', (512, 512), color=(73, 109, 137))
        img.save(path)

    return path

```

app/layout_builder.py

```

def build_comic_layout(image_paths: list, full_story: str, outline: list) -> list:
    """
    Organizes panel artwork, narration, and outlines into a coherent layout dictionary list.
    """
    # Split story by Panel headings
    story_panels = full_story.split("**Panel")
    story_panels = [f"**Panel{panel}" for panel in story_panels if panel.strip()]

    layout = []
    for idx, (image, text, panel_info) in enumerate(zip(image_paths, story_panels, outline), start=1):
        layout.append({
            "panel": idx,
            "title": panel_info.get("title", f"Panel {idx}"),
            "image_path": image,
            "text": "\n".join(text.strip().splitlines()[1:]).strip() if len(text.strip().splitlines()) > 1
        else text.strip(),
            "scene_description": panel_info.get("scene_description", "")
        })

    return layout

```

app/exporters.py

```

import os
from datetime import datetime
from fpdf import FPDF

EXPORT_FOLDER = "static/exports"

def save_pdf(layout: list) -> str:
    """
    Compiles the full comic into a PDF document using FPDF.
    """
    pdf = FPDF()

```

```

pdf.set_auto_page_break(auto=True, margin=15)
os.makedirs(EXPORT_FOLDER, exist_ok=True)

for panel in layout:
    image_path = panel['image_path']
    story_text = panel['text']

    pdf.add_page()

    # Panel title
    pdf.set_font("Helvetica", 'B', 14)
    pdf.cell(0, 10, f"Panel {panel['panel']}: {panel['title']}", ln=True, align="C")
    pdf.ln(5)

    # Image placement
    y_image = 30
    image_height = 100

    if os.path.exists(image_path):
        pdf.image(image_path, x=25, y=y_image, w=160, h=image_height)
    else:
        pdf.set_y(y_image)
        pdf.set_font("Helvetica", 'I', 10)
        pdf.multi_cell(0, 10, f"Image missing: {image_path}")

    # Text placement below image
    pdf.set_y(y_image + image_height + 15)
    pdf.set_font("Helvetica", '', 11)

    cleaned_text = story_text.replace("***", "")
    pdf.multi_cell(0, 8, cleaned_text)

    timestamp = datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"comic_{timestamp}.pdf"
    pdf_path = os.path.join(EXPORT_FOLDER, filename)
    pdf.output(pdf_path)

    return pdf_path

```

app/routes.py

```

from fastapi import APIRouter, Request, Form, HTTPException
from fastapi.responses import HTMLResponse, JSONResponse
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel
import traceback

from app.gemini_flash import generate_outline
from app.gemini_pro import generate_story
from app.image_generator import generate_image
from app.layout_builder import build_comic_layout
from app.exporters import save_pdf

router = APIRouter()
templates = Jinja2Templates(directory="templates")

class PromptRequest(BaseModel):
    prompt: str
    character_name: str
    setting: str
    tone: str
    style: str

@router.get("/", response_class=HTMLResponse)
async def index(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

@router.post("/generate", response_class=HTMLResponse)
async def generate_comic(
    request: Request,
    prompt: str = Form(...),

```

```

character_name: str = Form(...),
setting: str = Form(...),
tone: str = Form(...),
style: str = Form(...)
):
    try:
        full_prompt = (
            f"{prompt}\n"
            f"The main character is {character_name}. "
            f"The setting is a {setting}. "
            f"The tone is {tone}. The art style is {style}."
        )

        outline = generate_outline(full_prompt)

        if not isinstance(outline, list) or not all("image_prompt" in panel for panel in outline):
            raise ValueError("Invalid outline structure from Gemini response.")

        full_story = generate_story(outline)
        images = [generate_image(panel["image_prompt"]) for panel in outline]
        layout = build_comic_layout(images, full_story, outline)

        pdf_path = save_pdf(layout)
        web_pdf_path = "/" + pdf_path.replace("\\", "/")

        return templates.TemplateResponse("comic_preview.html", {
            "request": request,
            "layout": layout,
            "pdf_path": web_pdf_path
        })

    except Exception as e:
        traceback.print_exc()
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/generate-comic/json")
async def generate_comic_json(payload: PromptRequest):
    try:
        full_prompt = f"{payload.prompt}. Main character: {payload.character_name}, Setting: {payload.setting}, Tone: {payload.tone}, Style: {payload.style}"
        outline = generate_outline(full_prompt)
        full_story = generate_story(outline)
        images = [generate_image(panel["image_prompt"]) for panel in outline]
        layout = build_comic_layout(images, full_story, outline)
        pdf_path = save_pdf(layout)

        return JSONResponse({
            "status": "success",
            "layout": layout,
            "pdf_path": pdf_path
        })

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/export-success", response_class=HTMLResponse)
async def export_success(request: Request, pdf_path: str = ""):
    return templates.TemplateResponse("export_success.html", {
        "request": request,
        "pdf_path": pdf_path
    })

@router.get("/test-image")
async def test_image(prompt: str = "A futuristic city at sunset, sci-fi, cinematic, artstation"):
    try:
        image_path = generate_image(prompt)
        return {"message": "Image generated successfully", "path": image_path}

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

4. HTML Frontend Templates

templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>ComicCraft - Create Your Comic</title>
  <style>
    body { font-family: Arial, sans-serif; background-color: #f0f4f8; margin: 0; padding: 40px; }
    .card { max-width: 600px; margin: 0 auto; background: white; padding: 30px; border-radius: 12px; box-
shadow: 0 4px 15px rgba(0,0,0,0.1); }
    h1 { text-align: center; color: #1e3a8a; }
    .form-group { margin-bottom: 15px; }
    label { font-weight: bold; display: block; margin-bottom: 5px; }
    input, select, textarea { width: 100%; padding: 10px; border: 1px solid #cbd5e1; border-radius: 6px;
box-sizing: border-box; }
    button { width: 100%; padding: 12px; background: #2563eb; color: white; font-weight: bold; border:
none; border-radius: 6px; cursor: pointer; font-size: 16px; }
    button:hover { background: #1d4ed8; }
  </style>
</head>
<body>
  <div class="card">
    <h1>🦊 Create Your Comic</h1>
    <form action="/generate" method="post">
      <div class="form-group">
        <label>Story Prompt:</label>
        <textarea name="prompt" rows="3" required>A brave fox explores an enchanted forest.</
textarea>
      </div>
      <div class="form-group">
        <label>Main Character Name:</label>
        <input type="text" name="character_name" value="Free" required>
      </div>
      <div class="form-group">
        <label>Setting:</label>
        <select name="setting">
          <option value="Forest">Forest</option>
          <option value="Space">Space</option>
          <option value="School">School</option>
          <option value="City">City</option>
        </select>
      </div>
      <div class="form-group">
        <label>Story Tone:</label>
        <select name="tone">
          <option value="Dramatic">Dramatic</option>
          <option value="Funny">Funny</option>
          <option value="Light-hearted">Light-hearted</option>
        </select>
      </div>
      <div class="form-group">
        <label>Art Style:</label>
        <select name="style">
          <option value="Realistic">Realistic</option>
          <option value="Comic Book">Comic Book</option>
          <option value="Anime">Anime</option>
        </select>
      </div>
      <button type="submit">Generate Comic</button>
    </form>
  </div>
</body>
</html>
```

templates/comic_preview.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Your Comic Preview</title>
  <style>
    body { font-family: Arial, sans-serif; background-color: #f8fafc; margin: 0; padding: 20px; }
    .container { max-width: 800px; margin: 0 auto; }
    .panel { background: white; padding: 20px; border-radius: 8px; margin-bottom: 20px; box-shadow: 0 2px
8px rgba(0,0,0,0.1); }
    .panel img { max-width: 100%; height: auto; border-radius: 6px; }
    .download-btn { display: block; width: 250px; margin: 30px auto; padding: 12px; text-align: center;
background: #0284c7; color: white; text-decoration: none; border-radius: 6px; font-weight: bold; }
  </style>
</head>
<body>
  <div class="container">
    <h1 style="text-align:center;">📖 Your Comic Preview</h1>
    {% for panel in layout %}
      <div class="panel">
        <h2>Panel {{ panel.panel }}: {{ panel.title }}</h2>
        
        <p><i>{{ panel.scene_description }}</i></p>
        <p>{{ panel.text }}</p>
      </div>
    {% endfor %}
    <a class="download-btn" href="/export-success?pdf_path={{ pdf_path }}">Download Your Comic as PDF</a>
  </div>
</body>
</html>

```

templates/export_success.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Comic Exported Successfully</title>
  <style>
    body { font-family: Arial, sans-serif; background-color: #f0fdf4; text-align: center; padding: 80px
20px; }
    .box { background: white; max-width: 500px; margin: 0 auto; padding: 40px; border-radius: 12px; box-
shadow: 0 4px 12px rgba(0,0,0,0.05); }
    h1 { color: #16a34a; }
    a { display: inline-block; margin-top: 20px; padding: 10px 20px; background: #2563eb; color: white;
border-radius: 6px; text-decoration: none; }
  </style>
</head>
<body>
  <div class="box">
    <h1>✅ Comic Exported Successfully!</h1>
    <p>Your AI-powered comic has been successfully created! Dive back in and bring another story to
life.</p>
    {% if pdf_path %}
      <p><a href="{{ pdf_path }}" download style="background:#16a34a;">📄 Save PDF File</a></p>
    {% endif %}
    <a href="/">🏠 Go Create Another Comic</a>
  </div>
</body>
</html>

```